Finance Tracker

A Role-Based Personal Finance Management System

Built with Python · Flask · JSON

Project Submission — 2025

Project Overview

Finance Tracker is a full-stack web application built with Python and Flask that enables multi-role personal finance management. The system supports three distinct user roles — Admin, Analyst, and User — each with a tailored dashboard and access privileges. Data is persisted in a structured JSON file, and all financial operations are automatically logged to a per-user passbook.

Key Highlights

- Role-based access control with three permission levels
- Complete transaction management with income and expense tracking
- Automatic passbook generation and running balance calculation
- Monthly breakdown chart visualising net balance trends
- Admin panel for user lifecycle management (create, update, delete)
- Analyst panel for cross-user transaction filtering and summary reporting
- Responsive dark-themed UI built with vanilla HTML, CSS, and Jinja2 templates

---

User Roles & Permissions

The system enforces strict role separation. Each role is authenticated via phone number and password and routed to its dedicated dashboard on login.

Role	Dashboard	Capabilities
Admin	/admin	Create, update, and delete user records. Manages the entire user base.
Analyst	/analyst	Filter and summarise transactions across all user accounts by date, category, or type.
User	/user	View own transactions, add income/expense entries, delete transactions, view passbook and monthly summary.

Project Architecture

Directory Structure

```
finance_tracker/
├── app.py                  # Flask application & all route definitions
├── data.json               # Persistent JSON data store
├── templates/
```

```
│   ├── login.html          # Login page
│   ├── user.html           # User dashboard
│   ├── admin.html          # Admin panel
│   └── analyst.html        # Analyst panel
└── utils/
    ├── core.py             # Data I/O, passbook, summaries, helpers
    ├── viewers.py          # View functions (transactions, passbook, summary)
    ├── transactions.py     # Add & delete transaction logic
    ├── records_update.py   # Person CRUD (create, update, delete)
    └── filters.py          # Analyst filtering & summary aggregation
```

Data Model

All data is stored in a single data.json file as a list of user objects. Each user has the following structure:

Field	Type	Description
Name	string	Full name of the account holder
DOB	string	Date of birth in YYYY-MM-DD format
Phone	string	10-digit phone number (primary identifier)
Password	string	Account password
Role	string	One of: admin, analyst, user
Transactions	array	List of transaction objects (TxnIndex, Amount, Type, Category, Date, Notes)
Summary	object	Auto-computed totals: TotalIncome, TotalExpenses, CurrentBalance, CategoryBreakdown, MonthlyTotals
Passbook	array	Audit log of all actions with Timestamp, Action, and Details

---

Module Reference

core.py

Central utility module providing data persistence and shared helpers.

Function	Description
load_data()	Loads and returns the JSON dataset from data.json
save_data(data)	Serialises and writes the dataset back to data.json
update_passbook(db, action, index, details)	Appends a timestamped audit entry to a user's Passbook

Function	Description
refresh_summary(db, index)	Recomputes and stores TotalIncome, TotalExpenses, CurrentBalance, CategoryBreakdown, and MonthlyTotals for a user
sum_transactions(db, index)	Returns (income, expense, balance) tuple for a user
category_summary(db, index)	Returns a dict of net amounts grouped by category
monthly_total(db, index)	Returns a dict of net amounts grouped by YYYY-MM month key
recent_transactions(db, index, n)	Returns the n most recent transactions sorted by date
log_action(db, index, action, details)	Calls update_passbook then saves data — used in routes to avoid duplicate logging

viewers.py

Read-only functions that format data for template rendering.

Function	Description
view_transactions(db, index)	Returns the full transactions list for a user
view_passbook(db, index)	Returns the passbook entries list (or empty list if none)
view_summary(db, index)	Computes and returns a summary dict with keys: TotalIncome, TotalExpense, CurrentBalance, CategorySummary, MonthlySummary
view_records(db, name, dob, phone)	Searches and returns matching user records (used for lookup)

transactions.py

Handles creation and deletion of individual transaction records.

Function	Description
new_transaction(db, index, category, amount, notes, txn_type, date)	Creates a transaction dict, appends it to the user's Transactions list, updates the passbook, refreshes the summary, and saves.
delete_transaction(db, index, txn_id)	Removes a transaction by its list index, updates the passbook, refreshes the summary, and saves.

records_update.py

Handles user record lifecycle management.

Function	Description
new_person(name, dob, phone, password, db)	Creates a new user record with empty Transactions, Summary, and Passbook. Saves data and logs the creation.
update_records(field, value, index, db)	Maps a field name (name, dob, phone_number) to its JSON key, updates the value, logs the change, and saves.
delete_person(name, dob, phone, db)	Matches all three fields before deleting — requires exact match. Logs deletion and saves.

filters.py

Analyst-specific filtering and aggregation utilities.

Function	Description
filter_transactions(db, index, start_date, end_date, category, txn_type, formatted)	Returns transactions matching all provided criteria. Any parameter can be None or empty to skip that filter.
filter_summary(transactions, formatted)	Computes TotalIncome, TotalExpenses, and CurrentBalance from a filtered transaction list.

API Routes

Method	Route	Role Required	Description
GET / POST	/	—	Login page. Authenticates by phone + password and redirects based on role.
GET	/user	Any	User dashboard. Renders transactions, passbook, summary, and recent activity.
POST	/add_transaction	Any	Adds a new transaction for the logged-in user.
GET	/delete_transaction/<id>	Any	Deletes a transaction by index for the logged-in user.
GET	/admin	admin	Admin panel. Lists all users.
POST	/admin/create	admin	Creates a new user record.
POST	/admin/update	admin	Updates a field on an existing user record.
POST	/admin/delete	admin	Deletes a user record after verifying name, DOB, and phone.
GET	/analyst	analyst	Analyst dashboard.

Method	Route	Role Required	Description
POST	/filter	analyst	Filters and summarises transactions for a given phone number.
GET	/logout	Any	Clears the session and redirects to login.

Setup & Installation

Prerequisites

- Python 3.8+ *(required)*
- Flask `pip install flask`

Running the Application

Clone or download the project, then run:

```
cd finance_tracker
pip install flask
python app.py
```

The app will automatically open in your default browser at http://127.0.0.1:5000. The JSON data file (data.json) will be created automatically on first run if it does not exist.

Default Credentials

An admin or analyst account must be seeded directly into data.json to bootstrap the system. Example:

```
{
    "Name": "Admin User",
    "DOB": "2000-01-01",
    "Phone": "9999999999",
    "Password": "admin123",
    "Role": "admin",
    "Transactions": [],
    "Summary": {},
    "Passbook": []
}
{
    "Name": "Analyst User",
    "DOB": "2000-01-01",
    "Phone": "9999999999",
    "Password": "analyst123",
    "Role": "admin",
```

```
    "Transactions": [],
    "Summary": {},
    "Passbook": []
}
```

UI Design

All templates share a unified dark-themed design system built with vanilla CSS and Jinja2. The interface is responsive and uses a consistent visual language across all pages.

Design Element	Detail
Color scheme	Dark navy background (#070D1A) with teal/blue gradient accents
Typography	Playfair Display (serif) for headings, DM Sans for body text — loaded via Google Fonts
Animations	Slide-up card entry animations, fade-in for flash messages
Flash messages	Colour-coded: green for success, red for danger, blue for info
Charts	Chart.js bar chart for monthly breakdown (green = surplus, red = deficit)
Forms	Inline validation with maxlength and pattern attributes on all phone inputs
Responsiveness	CSS Grid with auto-fit columns; breakpoints at 900px and 600px

Design Decisions & Notes

Logging Strategy

Each route calls log_action (not update_passbook directly) to ensure data is always saved after logging. Functions in utils that internally call update_passbook do not call save_data for passbook entries — save_data is only called once at the end of each operation to avoid race conditions and duplicate writes.

Deletion Safety

The delete_person function requires an exact match on all three fields — Name, DOB, and Phone — before removing a record. This prevents accidental deletion from a phone-number-only lookup.

Summary Refresh

Summaries are not stored permanently and are recomputed by refresh_summary after every transaction change. The view_summary function in viewers.py computes summary data live from transactions, so even if the stored Summary object is stale, the rendered values are always accurate.

Passbook vs Log

The Passbook array serves as both a user-facing audit trail and a developer log. Entries record every action from transaction additions to admin operations, giving a full audit history per user account.

Technology Stack

Layer	Technology	Purpose
Backend	Python 3 + Flask	Web framework, routing, session management, Jinja2 templating
Data store	JSON (data.json)	Lightweight file-based persistence — no database required
Frontend	HTML5 + CSS3	Responsive dark-themed UI with CSS Grid and custom properties
Templating	Jinja2	Dynamic HTML rendering with conditionals, loops, and filters
Charts	Chart.js 4.4.1	Monthly breakdown bar chart via CDN
Fonts	Google Fonts	Playfair Display + DM Sans loaded via CDN